

Trabajo Práctico 1: Registro de préstamos de la biblioteca

Objetivo

El objetivo de este trabajo práctico es:

- Aplicar el concepto de **TDA**: separar el **contrato** de la **implementación**.
- Implementar un TDA **sobre arreglos**, manejando el crecimiento de la estructura a mano.
- Definir **invariantes** y validarlas en el constructor, y escribir **excepciones propias**.
- Leer y escribir **archivos de texto** con `java.nio.file.Files`.
- Escribir **pruebas automáticas** con JUnit 5.

Se debe desarrollar un programa por consola que procese el archivo de préstamos de la biblioteca de la facultad, lo valide, y produzca dos reportes: el **detalle de multas por socio** y el **ranking de títulos más pedidos**. El programa recibe el archivo de entrada y escribe los reportes en disco.

Reglas del negocio

- 1) El plazo de préstamo es de **14 días corridos** desde el retiro.
- 2) Un préstamo **sin fecha de devolución** todavía está en poder del socio.
- 3) Los **días de atraso** se cuentan desde el vencimiento (retiro + 14 días) hasta la fecha de devolución. Si el préstamo está pendiente, se cuentan hasta la **fecha de corte** que recibe el programa. Si no hay atraso, son **0 días**, nunca un número negativo.
- 4) La **multa** es de **\$150 por día de atraso**, con un **tope de \$3000 por préstamo**.
- 5) Un socio queda en estado **CON_DEUDA** si su multa acumulada es mayor a 0; si no, **AL_DIA**.

Formato del archivo de entrada

Archivo de texto, un préstamo por línea, con los campos separados por **punto y coma**:

```
fechaRetiro;padron;socio;isbn;título;fechaDevolucion
```

- Las fechas van en formato **ISO** (AAAA-MM-DD), que es lo que `LocalDate.parse` entiende sin configurar nada.
- El campo `fechaDevolucion` **puede venir vacío** (préstamo pendiente).
- Las líneas **en blanco** y las que empiezan con `#` se ignoran: no cuentan como error.
- El separador es `;` y no `,` a propósito, porque hay títulos que contienen comas.

El archivo de prueba se entrega junto con este enunciado. Sus primeras líneas son:

```
# Biblioteca FIUBA - registro de prestamos
# formato: fechaRetiro;padron;socio;isbn;título;fechaDevolucion

2026-03-02;41234;Ana Gomez;9789871234567;Estructuras de Datos;2026-03-16
2026-03-05;39876;Bruno Ferrari;9780262033848;Introduction to Algorithms;2026-03-30
2026-03-09;42001;Carla Nunez;9789871234567;Estructuras de Datos;2026-03-20
2026-03-11;40555;Diego Ruiz;9788478290499;El Lenguaje de Programacion C;
```

Sobre el final, el archivo trae **cuatro líneas deliberadamente inválidas**, una de cada tipo de error que hay que detectar:

Línea del archivo	Problema que tiene
2026-04-30;41234;Ana Gomez;9789871234567	Faltan campos (llegan 4 de 6)
2026-13-02;39876;Bruno Ferrari;...	Mes 13: fecha inválida
2026-04-18;CUARENTA;Carla Nunez;...	El padrón no es numérico
2026-04-15;43310;Elena Sosa;...;2026-04-01	Devolución anterior al retiro

El programa no debe abortar cuando encuentra una línea inválida. La descarta, guarda el motivo junto con el número de línea, y sigue con las demás. Al terminar informa cuántas descartó y por qué.

Clases requeridas

El programa debe estar dividido en al menos **6 tipos**. Todos van en el mismo paquete.

1. Clase Prestamo

Representa un préstamo. Es una **clase de valor**: dos préstamos con los mismos datos son el mismo préstamo, así que conviene un record, que ya trae equals, hashCode y toString.

```
public record Prestamo(LocalDate retiro, int padron, String socio,
                      String isbn, String titulo, LocalDate devolucion) {

    // Valida en el constructor compacto: un Prestamo mal formado NO debe existir.
    // - socio, isbn y titulo no pueden ser null ni vacios
    // - padron debe ser positivo
    // - devolucion puede ser null (pendiente), pero si no lo es,
    //   no puede ser anterior a retiro

    public boolean estaPendiente() { ... }
    public LocalDate vencimiento() { ... } // retiro + 14 dias
    public int diasDeAtraso(LocalDate corte) { ... } // siempre >= 0
    public int multa(LocalDate corte) { ... } // 150 por dia, tope 3000
}
```

2. Interfaz RegistroDePrestamos

Es el **contrato** del TDA. Describe qué se puede hacer con un registro de préstamos, sin decir en ningún lado cómo se guardan los datos.

```
public interface RegistroDePrestamos {
    void registrar(Prestamo p);
    int cantidad();
    Prestamo obtener(int i); // IndexOutOfBoundsException si i es invalido
    int[] padrones(); // sin repetidos, en orden de aparicion
    Prestamo[] prestamosDe(int padron); // arreglo vacio si no hay ninguno
    String[] titulosMasPedidos(int n); // los n mas pedidos, desempate alfabetico
}
```

3. Clase RegistroSobreArreglo

Es la **implementación** del contrato anterior, sobre un arreglo interno que ustedes hacen crecer a mano:

- arranca con capacidad 8;
- cuando se llena, se crea un arreglo del **doblo** de capacidad y se copian los elementos (Arrays.copyOf está permitido);
- cantidad() devuelve la cantidad de elementos **usados**, no la capacidad del arreglo.

Este punto es el corazón del trabajo práctico. Si lo resuelven con ArrayList, el TP no cumple su objetivo.

4. Clase LectorDePrestamos

Lee el archivo con `Files.readAllLines`, ignora blancos y comentarios, arma un Prestamo por cada línea válida, y por cada línea inválida guarda un mensaje con el **número de línea real del archivo** (contando comentarios y blancos).

```
public class LineaInvalidaException extends RuntimeException {
    public LineaInvalidaException(int numeroDeLinea, String motivo) { ... }
    public int numeroDeLinea() { ... }
}

public record ResultadoDeCarga(RegistroDePrestamos registro,
                               String[] errores, int lineasDeDatos) { }

public class LectorDePrestamos {
    public static ResultadoDeCarga cargar(Path archivo) throws IOException { ... }
}
```

5. Clase Reporteador y los exportadores

El reporteador arma las filas del reporte; los exportadores las escriben en disco. Las filas van ordenadas por **multa descendente** y, a igual multa, por **nombre de socio alfabético**.

```
public record FilaDeSocio(int padron, String socio, int prestamos,
                          int diasDeAtraso, int multa, String estado) { }

public class Reporteador {
    public static FilaDeSocio[] porSocio(RegistroDePrestamos r, LocalDate corte) { ... }
    public static String[] ranking(RegistroDePrestamos r, int n) { ... }
}

public interface ExportadorDeReporte {
    void exportar(FilaDeSocio[] filas, Path destino) throws IOException;
    String extension(); // "txt", "csv", ...
}
```

Hay que implementar **dos** exportadores: `ExportadorTxt` y `ExportadorCsv`. El programa principal **no debe saber cuál de los dos está usando**: recibe un `ExportadorDeReporte` y lo usa. Poder cambiar el formato de salida sin tocar el resto del programa es exactamente para lo que sirve separar el contrato de la implementación.

6. Clase Tp1

Contiene el `main()`. Recibe por parámetro el archivo de entrada y la fecha de corte; si no se los pasan, usa los valores por defecto, de manera que corriendo el programa sin argumentos se obtenga exactamente la salida de la sección siguiente.

```
public class Tp1 {
    private static final String ENTRADA_POR_DEFECTO = "datos/prestamos.csv";
    private static final LocalDate CORTE_POR_DEFECTO = LocalDate.parse("2026-05-04");

    public static void main(String[] args) throws IOException {
        // args[0] = archivo de entrada (opcional)
        // args[1] = fecha de corte ISO (opcional)
    }
}
```

Usá siempre **rutasy relativas**, nunca absolutas (Apéndice A, punto 2).

Ejemplo de uso

Con el archivo de prueba entregado y la fecha de corte 2026-05-04, la salida debe ser **exactamente** esta. Si les da distinto, algo está mal.

Resumen de la carga, por consola

```
Lineas de datos: 22 | validas: 18 | descartadas: 4
linea 28: se esperaban 6 campos y llegaron 4
linea 29: fecha invalida: 2026-13-02
linea 30: padron no numerico: CUARENTA
```

línea 31: la devolución (2026-04-01) es anterior al retiro (2026-04-15)

El texto del motivo puede variar; el número de línea y las cantidades, no.

Archivo reporte.txt

BIBLIOTECA FIUBA - REPORTE DE MULTAS
Fecha de corte: 2026-05-04

Padron	Socio	Prestamos	DiasAtraso	Multa	Estado
39876	Bruno Ferrari	4	24	3600	CON_DEUDA
41234	Ana Gomez	4	23	3450	CON_DEUDA
40555	Diego Ruiz	3	42	3300	CON_DEUDA
43310	Elena Sosa	3	10	1500	CON_DEUDA
42001	Carla Nunez	4	0	0	AL_DIA
TOTALES		18	99	11850	

TITULOS MAS PEDIDOS

- | | |
|-------------------------------|---|
| 1. Estructuras de Datos | 6 |
| 2. Clean Code | 3 |
| 3. Introduction to Algorithms | 3 |

Fíjense en el desempate del ranking: *Clean Code* e *Introduction to Algorithms* tienen 3 préstamos cada uno, y va primero el alfabéticamente menor. Para las columnas de ancho fijo usen `String.format`.

Archivo reporte.csv

```
padron;socio;prestamos;dias_atraso;multa;estado
39876;Bruno Ferrari;4;24;3600;CON_DEUDA
41234;Ana Gomez;4;23;3450;CON_DEUDA
40555;Diego Ruiz;3;42;3300;CON_DEUDA
43310;Elena Sosa;3;10;1500;CON_DEUDA
42001;Carla Nunez;4;0;0;AL_DIA
```

Este archivo lo abre Excel con doble click. Se usa `;` y no `,` porque en la configuración regional de Argentina Excel espera el punto y coma como separador de campos: la coma es el separador decimal.

Requisitos mínimos

El programa debe:

- Compilar y ejecutarse correctamente.
- Tener al menos los 6 tipos pedidos, cada uno en su propio archivo.
- Leer el archivo de entrada y escribir los dos reportes en disco.
- Descartar las líneas inválidas sin cortar la ejecución, e informarlas.
- Producir exactamente los valores del ejemplo de uso.
- Soportar archivos de más de 8 préstamos, haciendo crecer el arreglo interno.

Restricciones

Para que el trabajo práctico tenga sentido, **está prohibido** usar:

- `ArrayList`, `HashMap`, `HashSet` ni ninguna colección de `java.util`. Todo se resuelve con **arreglos**. Sí se pueden usar `Arrays.copyOf`, `Arrays.sort` y `Arrays.fill`.
- La API de Streams (`.stream()`, `.map()`, `.collect()`).
- Librerías externas, **salvo** para el punto adicional opcional descripto más abajo.

Sí se pueden y se deben usar: `String` y sus métodos, `StringBuilder`, `LocalDate`, `ChronoUnit`, `Files`, `Path`, `Math`, `String.format`, `record` y `switch`.

Interfaz de usuario

Toda la interfaz debe estar basada en texto. El programa no lleva menú: se ejecuta, procesa el archivo, imprime el resumen de la carga por pantalla y escribe los reportes en disco. No es necesario limpiar la pantalla.

Pruebas automáticas

Se deben entregar **como mínimo 12 pruebas** con JUnit 5, que cubran:

- **Prestamo**: que cada validación del constructor rechace los datos malos (`assertThrows`), el cálculo del vencimiento, atraso 0 cuando se devolvió en fecha, atraso de un préstamo pendiente, y el tope de multa en 3000.
- **RegistroSobreArreglo**: que crezca más allá de la capacidad inicial (registrar 20 préstamos y verificar que `cantidad()` sea 20), `padrones()` sin repetidos, `prestamosDe` con un padrón inexistente devuelve arreglo vacío, y el desempate alfabético del ranking.
- **LectorDePrestamos**: sobre el archivo de prueba, 18 líneas válidas y 4 errores.
- **ExportadorCsv**: escribir a un archivo temporal (`Files.createTempFile`) y volver a leerlo con `Files.readAllLines` para comparar contra el contenido esperado.

Punto adicional opcional (+1 punto): exportar a Excel

Quien lo desee puede sumar **1 punto** a la nota final implementando un tercer exportador, `ExportadorXlsx`, que genere una planilla `.xlsx` nativa (encabezado en negrita y columnas autoajustadas) usando la librería **Apache POI**.

Es **opcional y adicional**: el `.csv` se entrega igual. Un trabajo que reemplace el CSV por el Excel **no** suma el punto.

Se declara la dependencia en `app/build.gradle`:

```
dependencies {  
    // ... lo que ya estaba ...  
    implementation 'org.apache.poi:poi-ooxml:5.5.1'  
  
    // Sin estas dos líneas POI imprime, al arrancar, el mensaje  
    // "ERROR Log4j API could not find a logging provider".  
    // No es un error del programa: es POI buscando un logger.  
    runtimeOnly 'org.apache.logging.log4j:log4j-to-slf4j:2.24.3'  
    runtimeOnly 'org.slf4j:slf4j-nop:2.0.16'  
}
```

La primera compilación va a descargar unos 19 MB en 16 archivos `.jar`. Es normal, y queda en la cache de Gradle: pasa una sola vez.

Con la librería de la cátedra

La cátedra provee `ar.uba.fi.cb100.librerias.excel.ExcelUtils`, que envuelve a POI y se usa igual que `Files`: métodos estáticos que reciben un `Path`. Con eso, el exportador entero es una línea:

```
String[] encabezados = {"Padron", "Socio", "Prestamos", "DiasAtraso", "Multa", "Estado"};  
Object[][] datos = new Object[filas.length][];  
for (int f = 0; f < filas.length; f++) {  
    FilaDeSocio fila = filas[f];  
    datos[f] = new Object[]{fila.padron(), fila.socio(), fila.prestamos(),  
                           fila.diasDeAtraso(), fila.multa(), fila.estado()};  
}  
  
ExcelUtils.escribir(destino, "Multas", encabezados, datos);
```

Los valores se escriben **con su tipo**: los padrones y las multas quedan como números, no como texto, así que Excel los suma y los ordena bien. Para leer una planilla, `ExcelUtils.leer(origen)` devuelve un `String[][]`, donde la fila 0 es el encabezado.

Directamente con Apache POI

Quien prefiera no usar la librería y trabajar contra POI, el esqueleto es este:

```

try (Workbook libro = new XSSFWorkbook()) {
    Sheet hoja = libro.createSheet("Multas");

    CellStyle negrita = libro.createCellStyle();
    Font fuente = libro.createFont();
    fuente.setBold(true);
    negrita.setFont(fuente);

    String[] encabezados = {"Padron", "Socio", "Prestamos",
                            "DiasAtraso", "Multas", "Estado"};
    Row cabecera = hoja.createRow(0);
    for (int c = 0; c < encabezados.length; c++) {
        Cell celda = cabecera.createCell(c);
        celda.setCellValue(encabezados[c]);
        celda.setCellStyle(negrita);
    }

    // ... una fila por cada FilaDeSocio, con hoja.createRow(i) ...

    for (int c = 0; c < encabezados.length; c++) {
        hoja.autoSizeColumn(c);
    }

    try (FileOutputStream salida = new FileOutputStream(destino.toFile())) {
        libro.write(salida);
    }
}

```

Para el punto adicional da lo mismo cuál de los dos caminos elijan.

Cuestionario

Responder el siguiente cuestionario:

- 1) ¿Qué es un TDA? ¿Por qué conviene separar el contrato (la interfaz) de la implementación?
- 2) ¿Qué es una invariante de clase? ¿Por qué conviene validarla en el constructor y no al principio de cada método?
- 3) Su implementación duplica la capacidad del arreglo cada vez que se llena. Al registrar N préstamos, ¿cuántas copias de elementos se hacen en total? ¿Qué pasaría si en lugar de duplicar, el arreglo creciera de a un lugar por vez?
- 4) ¿Qué diferencia hay entre `==` y `equals` al comparar dos objetos `Prestamo`? ¿Por qué al usar un record el `equals` funciona sin escribirlo?
- 5) ¿Por qué hay que usar `split(";", -1)` y no `split(";")` para partir cada línea del archivo? ¿Qué préstamos se romperían si se usara la segunda forma?

Normas de entrega

Trabajo práctico **individual**: 1 persona.

Reglas generales: respetar el **Apéndice A**.

Se deberá subir un único archivo comprimido al campus, en un link que se habilitará para esta entrega. Este archivo deberá tener un nombre formado de la siguiente manera:

Padron-TP1.zip

Deberá contener los archivos fuentes (no los binarios), las pruebas de JUnit, el archivo de datos de prueba, los datos personales (padrón, nombre y mail), el informe del trabajo realizado, las respuestas al cuestionario, el manual del usuario y el manual del programador (todo en el mismo PDF).

La fecha de entrega vence el día << **COMPLETAR: viernes DD/MM/26** >> a las 23.59 hs por el campus.

Se evaluará: funcionalidad, eficiencia, algoritmos utilizados, buenas prácticas de programación, modularización, documentación, gestión de memoria y estructuras de datos.

Apéndice A

- 1) Usar las siguientes convenciones para nombrar identificadores.
 - a) **Clases:** Los nombres de clases siempre deben comenzar con la primera letra en mayúscula en cada palabra, deben ser simples y descriptivos. Se concatenan todas las palabras. Ejemplo: Coche, Vehiculo, CentralTelefonica.
 - b) **Métodos:** Deben comenzar con letra minúscula, y si está compuesta por 2 o más palabras, la primera letra de la segunda palabra debe comenzar con mayúscula. De preferencia que sean verbos. Ejemplo: arrancarCoche(), sumar().
 - c) **Variables y objetos:** las variables siguen la misma convención que los métodos. Por ejemplo: alumno, padronElectoral.
 - d) **Constantes:** Las variables constantes o finales, las cuales no cambian su valor durante todo el programa, se deben escribir en mayúsculas, concatenadas por "_". Ejemplo: ANCHO, VACIO, COLOR_BASE.
- 2) Si el trabajo práctico requiere archivos para procesar, entregar los archivos de prueba en la entrega del TP. Utilizar siempre **rutas relativas** y no absolutas.
- 3) Entregar el informe explicando el TP realizado, manual de usuario y manual del programador en un mismo PDF dentro del zip.
- 4) Comentar el código. Todos los tipos, métodos y funciones deberían tener sus comentarios.
- 5) Modularizar el código. No entregar 1 o 2 archivos, separar cada clase.
- 6) Si cualquier estructura de control tiene 1 línea, utilizar **{ }** siempre, por ejemplo:

```
for (int i = 0; i < 10; i++) {  
    System.out.println(i);  
}
```